

Echtzeitsysteme – Aufgabe 1: Pin Toggle

Periodische GPIO-Signalerzeugung auf dem Raspberry Pi 5 (Ubuntu) – Periode & Jitter

1. Aufbau und Vorgehen

Auf einem Raspberry Pi 5 unter Ubuntu wird mit `libgpiod` (v2) ein periodisches Rechtecksignal an GPIO26 (`gpiochip0`, Offset 26) erzeugt. Ein eigener Worker-Thread schaltet den Pin abwechselnd HIGH/LOW und wartet zwischen den Flanken mit `nanosleep()` jeweils eine halbe Periode. Direkt nach jedem Toggle wird mit `clock_gettime(CLOCK_MONOTONIC)` die seit dem letzten Toggle vergangene Zeit gemessen und in einen Ringpuffer geschrieben; ein zweiter Thread exportiert die Werte als CSV.

Getestet wurde mit einer Zielfrequenz von 100 Hz (Sollwert der halben Periode = 5.000.000 ns = 5 ms). Der Jitter ist definiert als Abweichung der gemessenen halben Periode vom Sollwert. Verglichen werden drei Konfigurationen:

Baseline – Standard-Scheduling (CFS), keine Bindung. **Core-Pinning** – Thread auf CPU-Kern 1 fixiert (-c 1). **RT** – zusätzlich SCHED_FIFO mit Priorität 80 (-c 1 -p 80).

2. Ergebnisse

Konfiguration	Samples	Ø halbe Periode	Std (Streuung)	Ø Jitter	Jitter p99	Worst-Case	eff. Freq.
Baseline	2.891	5.155.593 ns	350.719 ns	155.593 ns	1.856.182 ns	2.983.173 ns	96.98 Hz
Core-Pinning	2.332	5.100.069 ns	199.621 ns	100.069 ns	1.209.628 ns	2.971.625 ns	98.04 Hz
RT (FIFO/80)	3.695	5.018.741 ns	14.177 ns	18.741 ns	61.558 ns	210.657 ns	99.63 Hz

Tabelle 1: Statistische Kennwerte je Konfiguration (n = Anzahl Halbperioden, Sollwert 5.000.000 ns). Die RT-Zeile ist grün hervorgehoben.

Die RT-Konfiguration reduziert die Streuung des Jitters von 350.719 ns auf 14.177 ns – eine Verbesserung um den Faktor ~25. Der schlechteste gemessene Ausreißer sinkt von 2.983.173 ns (~3 ms!) auf 210.657 ns (Faktor ~14). Reines Core-Pinning hilft moderat, beseitigt die großen Ausreißer aber nicht – erst die Echtzeit-Priorität (SCHED_FIFO) verhindert, dass andere Prozesse den Thread verdrängen.

3. Diagramme

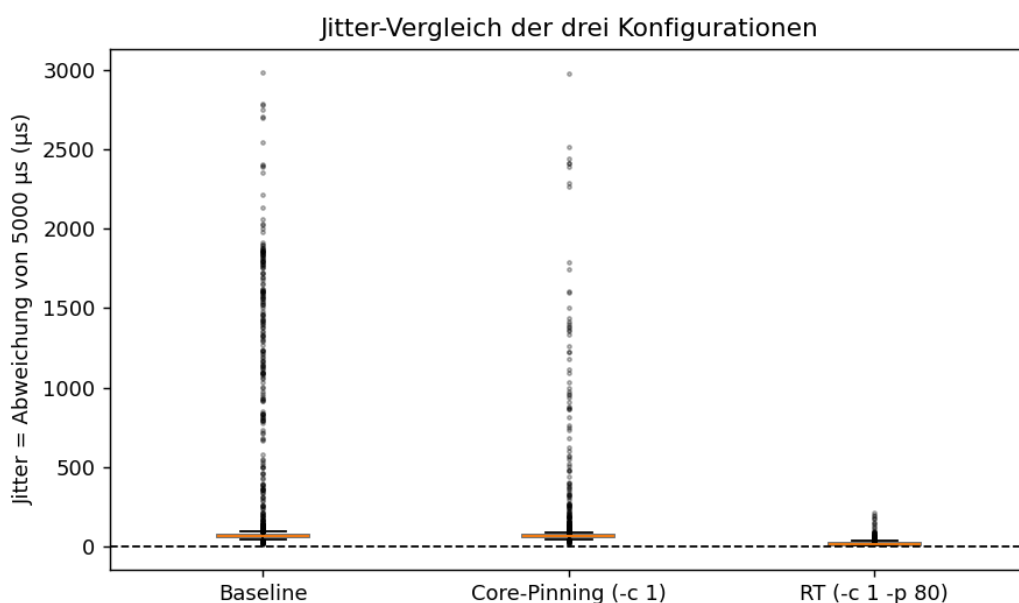


Abb. 1: Box-Plot des Jitters. Baseline und Core-Pinning zeigen viele und große Ausreißer nach oben; RT liegt eng um 0.

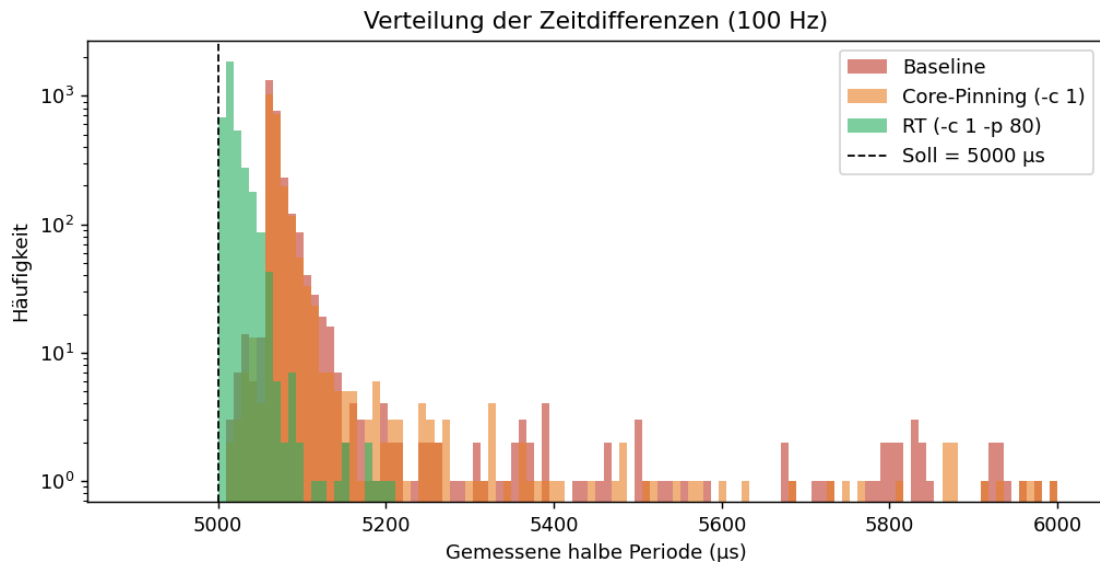


Abb. 2: Verteilung der gemessenen halben Periode (log. Häufigkeit). RT ist scharf um 5 ms konzentriert.

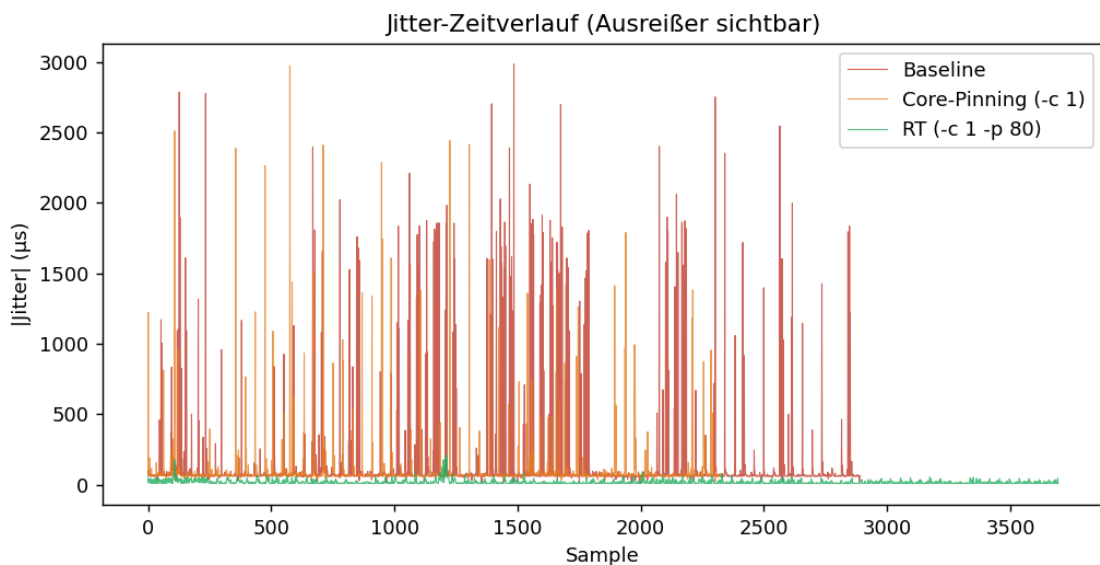


Abb. 3: Jitter-Zeitverlauf. Bei Baseline/Core treten sporadische Spikes bis ~3 ms auf, bei RT nicht.

4. Interpretation

Jitter: Im Standard-Scheduling wird der Thread regelmäßig von anderen Tasks und Kernel-Aktivität unterbrochen; das erzeugt die seltenen, aber großen Ausreißer (Worst-Case ~3 ms = 60 % der Periode). Mit SCHED_FIFO/Priorität 80 läuft der Thread bevorzugt und wird kaum verdrängt – Streuung und Worst-Case fallen drastisch.

Mittlere Periode: In allen Läufen liegt die halbe Periode systematisch leicht über 5 ms, die effektive Frequenz also knapp unter 100 Hz. Ursache ist das *relative nanosleep()*: Die Zeit für Toggle und Messung kommt pro Halbperiode oben drauf und summiert sich nicht weg. Bei der RT-Variante ist dieser Versatz am kleinsten (99,63 Hz), weil die Ausführung gleichmäßiger ist.

Worst-Cases: Die kritischen Werte sind nicht der Mittelwert, sondern die Ausreißer – für Echtzeitsysteme ist die garantierte obere Schranke entscheidend. Nur die RT-Konfiguration hält diese klein ($p99 \approx 62 \mu s$).

Mögliche weitere Verbesserung: Ein *clock_nanosleep()* mit absolutem Zeitpunkt (TIMER_ABSTIME) statt *relative nanosleep()* würde den systematischen Frequenzversatz beseitigen, da auf feste Zeitpunkte statt auf relative Intervalle gewartet wird. Zusätzlich helfen CPU-Isolation (*isolcpus*) und ein PREEMPT_RT-Kernel.